

Generation Is Solved

The new software development lifecycle, from vibe coding to agentic engineering



JOAKIM ABDINUR IUSUF

JUN 24, 2026

Last time, I ended by promising to write about a shift happening in software engineering — one that goes by the name *agentic engineering*. It distills the first whitepaper in Google's Vibe Coding series, *The New SDLC With Vibe Coding (May 2026)*. In my last article, I explained what an agent is. In this one, we're asking a harder question: now that agents can write the code, what happens to the people who used to?

For most of computing history, programming has been an act of translation. You understand a problem in human terms, design a solution in abstract terms, then grind it down into syntax a machine will accept. Every step loses something, and the whole craft was learning to lose as little as possible.

That translation layer is collapsing. Increasingly, developers say *what* they want and trust a system to handle the *how*. This isn't speculative: as of early 2026, around 85% of professional developers use AI coding agents, and roughly 41% of all new code is AI-generated. The interface to the machine is becoming intent.

The paper's argument, in one line: generation is no longer the hard part. The hard part (the part that is now the actual job) is verification, judgment, and direction. Everything below is an unpacking of that sentence.

Vibe coding is a spectrum, not a verdict

The phrase “vibe coding” comes from Andrej Karpathy, who in early 2025 described giving in to the vibes and forgetting the code even exists: you say what you want, paste the error back when it breaks, and let the model sort it out. It went viral because it named something people were already doing. Then it got applied to *everything*, until a senior engineer shipping a well-specified feature and a hobbyist hot-gluing prompts together were both “vibe coding.” The term stopped meaning anything. By 2026, Karpathy himself had introduced “agentic engineering” for the disciplined end of the range.

The useful reframe is to stop treating these as two camps and see them as endpoints on a spectrum. And the thing that places you on that spectrum is not whether you use AI, since everyone uses AI now. It is how much structure, verification, and human judgment wrap around what the AI produces.

If there’s a single idea to take from this, it’s that the spectrum is really about *verification*. At the disciplined end, two mechanisms work together. **Tests** check the deterministic parts: this input produces that output. **Evals** check the parts that aren’t deterministic (did the agent take a sensible path, choose the right tools, and produce something that clears the quality bar?), and they’re judged by labelled datasets, scoring rubrics, and other models acting as graders. Without both, it’s vibe coding, no matter how sophisticated the prompts look.

Where you should sit depends entirely on the stakes. A weekend prototype can be pure vibes. A payment system moving real money cannot. Most work lands somewhere in between, and the real skill is knowing where to draw the line for each task.

The real skill is context, not cleverness

Here’s the insight that took the field a while to internalise: the quality of AI-generated code depends far less on how cleverly you word the prompt than on the quality of the context you provide. The question

worth asking isn't "how do I trick the model into writing good code?" It's "what would a new teammate need to know to do this well, and how do I hand it to them in a form the model can use?"

That practice has a name now — context engineering — and it spans six kinds of context:

1. The agent's instructions.
2. Retrieved knowledge.
3. Memory.
4. Examples.
5. The tools it can call.
6. The guardrails it cannot cross.

The craft is deciding what the agent carries at all times (expensive, because every token is in the window on every turn) versus what it pulls in only when a task calls for it (cheap, on demand). Agent Skills are the cleanest pattern for the second kind: the agent stays a lightweight generalist and loads specialist knowledge only when it's needed.

Agent = Model + Harness

When an agent does something dumb, the reflex is to blame the model. Wrong model, bad day, wait for the next release. That instinct is almost always misplaced, and it leads to the wrong investments. The model is one input. Everything else — the prompts, the tools, the sandboxes, the orchestration logic, the guardrails, the observability — is the *harness*: the scaffolding that turns a raw model into something that can actually finish a job. In the whitepaper, they say that the model is roughly 10% of a working agent, and the harness is the other 90%.

An example: On a public coding benchmark, one team moved an agent from outside the top 30 into the top 5 by changing only the harness, with the same model underneath. The everyday version of that result is

worth memorising: *when an agent fails, the cause is usually a missing tool, a vague rule, an absent guardrail, or a context window stuffed with noise*. Most agent failures, looked at honestly, are configuration failures.

That single shift in attribution, from “the model isn’t good enough” to “my harness isn’t good enough”, is most of what separates people who get value from agents from people who keep getting frustrated by them.

You’re building the factory

The mental model that ties it all together is the *factory*. A factory manager doesn’t assemble each unit by hand; they design the line and own quality control. The modern developer’s primary output isn’t code anymore — it’s the *system that produces code*: the specs, the agents, the tests and quality gates, the feedback loops that route failures back for another pass, and the guardrails that keep everything inside the lines.

Inside that factory, your day tends to split into two modes. As a **conductor**, you’re hands-on and in real time, in the editor, watching code appear, correcting line by line (think GitHub Copilot). It’s the right mode for tricky logic and unfamiliar code, and it’s comfortable for anyone from a traditional engineering background; its weakness is that you become the bottleneck. As an **orchestrator**, you work a level up: define a task precisely, hand it to one or more agents running in the background, and review the result rather than the keystrokes (think Claude Code). That’s the right mode for well-specified work like bug fixes, migrations, and test generation. Most of us move between the two depending on the task.

Neither mode escapes what the paper calls the **80% problem**. Agents will get you roughly 80% of a feature fast. The remaining 20% (edge cases, integration points, the subtle correctness requirements that depend on knowing *the* ins-and-outs of a business/problem/codebase) is exactly where current models are weakest, and exactly where your

attention belongs. The failure mode of the moment isn't a syntax error; it's code that looks right, passes a basic test, and quietly encodes a wrong assumption. It's telling that one widely-cited study found experienced developers were actually *slower* on some tasks once you counted the time spent verifying and correcting the output. AI doesn't remove the implementation work so much as convert it from writing into reviewing.

When the thing you build is itself an agent

Everything so far has been about using coding agents to *build software*. But increasingly the thing you need to build is itself an agent — a support bot that handles refunds, a research assistant that produces grounded reports, an internal tool that watches for compliance anomalies. Those aren't scripts. They're products, with their own tools, memory, evaluation, and deployment to worry about.

The shift worth noticing is that the same terminal workflow now reaches all the way to those production agents. What used to mean a separate stack (frameworks, cloud consoles, a dedicated SDK) increasingly happens by talking to the same coding agent you were already using. Google's Agents CLI is built around exactly this: a small tool that teaches your coding agent (Claude Code, Codex, whichever) the full ADK lifecycle — scaffold the project, write the agent, generate an evalset, deploy it to a managed runtime, wire up observability — without you ever learning a new framework. You describe what you want; it does the right thing at each step. The prototype that ran on your laptop yesterday can become the agent serving real users today, without a rewrite.

And it scales past a single agent. ADK gives you multi-agent workflows out of the box, with coordination through shared state, MCP for tools, and A2A for handing work between agents. As an example, Anthropic's engineers had a team of agents build a working C compiler in Rust over two weeks, with humans setting direction and reviewing output but not

writing the implementation. The bottleneck moved from writing the code to specifying what it should do and checking that the agents did it — the whole thesis of this whitepaper, playing out at the scale of an entire compiler.

Why the discipline pays

It's tempting to read all this as "agentic engineering is just vibe coding with extra chores." The economics say otherwise. Vibe coding looks almost free (a subscription and a few prompts), but the bill arrives later as operational cost: tokens burned re-prompting the model to fix its own unverified mistakes, days lost reverse-engineering AI-generated spaghetti six months on, and security holes that are far more expensive to patch in production than to prevent in design. Agentic engineering inverts the curve: more invested upfront in schemas, tests, and context, far less paid out over the life of the system. From this angle, context engineering isn't only a technical skill — it's a financial lever, because every token you don't waste is money.

Where to start

None of this is a future state; it's the work in front of us today. And since AI amplifies whatever engineering culture it lands in, the practices that matter are just the ones that translate that into habits. If you're an individual builder, here's where I'd start:

1. **Write an** AGENTS.md (or CLAUDE.md, or whatever your tool reads). Ten lines is enough to begin (stack, conventions, hard rules it cannot break, workflow), and you add a new rule every time the agent does something you don't want repeated. That file *is* your harness.
2. **Write the tests and evals before the code.** Together, they're your contract with the AI: they pin down intent more precisely than any prompt, and they're the single cleanest thing that turns vibe coding

into agentic engineering. (Think Test-Driven Development, or as we'll see in the last whitepaper, Spec-Driven Development.)

3. **Pick one repetitive workflow and make it your first agent.** A recurring report, a review step, a research task. Prototype it with a coding agent and graduate it to a production agent (through something like Agents CLI) once it earns its keep. Building one end-to-end teaches more than reading a hundred articles like this one.
4. **Review every line that's going to ship.** Be suspicious of anything that looks too clever, check that the packages it imported actually exist, and make sure the error handling covers failures you'll really hit. Code no one on the team understands is debugging debt you can't afford.
5. **Keep your own fundamentals sharp.** The deal where AI takes the routine and you take the hard part only holds if your debugging, system design, and instinct for correctness stay strong. AI is a way to apply expertise at a greater scale, never a substitute for having it.

Which brings me back to where I started. For most of computing history, the bottleneck was getting the machine to do the thing at all. That bottleneck is dissolving. Generation is solved. Verification, judgment, and direction are the new craft. That, more than any model release, is the shift worth building your skills around.

Based on "The New SDLC With Vibe Coding" by Addy Osmani, Shubham Saboo, and Sokratis Kartakis (Google, May 2026).